

EI1061.ET-PR-2-2

TEMA II. PARALELISMO A NIVEL DE DATOS.

Parte II: Procesadores vectoriales

OBJETIVO

- Desarrollar código para el procesador vectorial RV64V con una longitud vectorial por defecto (64 o 32 elementos en función del tamaño de los datos)

PROBLEMAS

1.- Implementa el código escalar y vectorial para la operación daxpy en el procesador RV64G y RV64V respectivamente con vectores de dimensión N elementos de doble precisión.

a) Versión escalar en pseudocódigo:

Procesamiento $N \leq 64$ (tamaño por defecto de los vectores)

Código general en LA:

for ($i=0$; $i < n$; $i=i+1$)

$Y[i] = a * X[i] + Y[i];$

Preparando el Código para procesador vectorial:

Consta:

- Suponemos que N es un valor cualquiera
- $mvl = 32$ (número de elementos que por defecto procesa una operación vectorial, obtenido a partir del tamaño de los datos y de los registros utilizados)
- vl = registro que almacena el número de elementos que se procesan en una operación vectorial. Por defecto es mvl pero este valor se puede modificar. $vl < MVL$
- La instrucción `setvl t0,a0` carga en el registro vl y $t0$ el contenido de $a0$.

Código en LA:

$N \gg MVL$

$low = 0$

$bloques = n/mvl$

for ($j=0$; $j < N$; $j=j+mvl$) {

for ($i=low$ $i < low + vl$; $i++$)

$Y[i] = a * X[i] + Y[i]$

$low = low + vl$

$n = n - vl$

}

$vl = \min(n, mvl)$

for ($i=low$ $i < low + vl$; $i++$)

$Y[i] = a * X[i] + Y[i]$

2 registros vectoriales: Doble precisión.

$1024/2 = 512$ bytes por registro $\rightarrow 512/8$ bytes (64 bits) = 64 elementos por registro

//a0 contiene n

Código en RV64V

```
                vsetdcfg 2 DP    # Enable 2 64b DP Pt. registers
                fld f0,a        # Load scalar a

loop:   setvl t0,a0              # vlr = t0 = min(mvl,a0)

                vld v0,x5        # Load vector X
                vmul.vs v0,v0,f0    # Vector-scalar mult
                vld v1,x6        # Load vector Y
                vadd.vv v1,v0,v1    # Vector-vector add
                vst v1,x6        # Store the sum into Y
                slli t1,t0,3      # t1 = t0 * 8 (in bytes)
                add x5,x5,t1      # Increment pointer to X by vl*8
                add x6,x6,t1      # Increment pointer to Y by vl*8
                sub a0,a0,t0      # n -= vl (t0)
                bnez a0,loop      # Repeat if n != 0

                vdisable          # Disable vector regs}
```

3.- Implementa el siguiente código utilizando registros predicados. Suponed que los elementos son en coma flotante pero simple precisión, (N =128)

for (i = 0; i < N; i=i+1)

if (X[i] != 0)

X[i] = X[i] – Y[i];

2 registros $\rightarrow 1024/2 = 512$ byt

Use predicate register to “disable” elements:

```
vsetdcfg      2*FP32      # Enable 2 32b FP vector regs
vsetpcfgi     1           # Enable 1 predicate register
```

```
vld           v0,x5        # Load vector X into v0
vld           v1,x6        # Load vector Y into v1
fmv.d.x       f0,x0        # Put (FP) zero into f0
vpne          p0,v0,f0     # Set p0(i) to 1 if v0(i)!=f0
vsub.vv       v0,v0,v1     # Subtract under vector mask
vst           v0,x5        # Store the result in X
vdisable      # Disable vector registers
vpdisable     # Disable predicate registers
```

2.- Implementa una suma de dos matrices (elemento a elemento) en el procesador RV64V de dimensión NxM de simple precisión.

Código en LA:

For (i=0; i<n;i++)

```

For (j=0;j<m; j++)
    C[i,j] = A[i,j] + B[i,j];

```

```

For (i=0; i<n*m; i++)
    C[i] = A[i] + B[i];

```

bloques = $m \cdot n / \text{MVL}$

resto = $m \cdot n \% \text{MVL}$

Preparando el código para procesador vectorial:

La matriz tiene nxm elementos y se puede ver con un vector de nxm

```

                                vsetdcfg  3 SP          # Enable 3 32b Fl.Pt. registers
                                lw          x1,n          # Load scalar n
                                lw          x2,m          # Load scalar m
                                mult        x1,x1,x2      #total elementos a sumar
loop:                          setvl       t0,x1         # vl = t0 = min(mvl, ,x1)
                                vld         v0,x5         # Load matrix A
                                vld         v1,x6         # Load matriz B
                                vadd.vv     v1,v0,v1      # Vector-vector add
                                vst         v1,x7         # Store the sum matrix c
                                slli        t1,t0,2       # t1 = vl * 4 (in bytes)
                                add         x5,x5,t1      # Increment pointer to X by vl*4
                                add         x6,x6,t1      # Increment pointer to Y by vl*4
                                add         x7,x7,t1
                                sub         x1,x1,t0      # n -= vl (t0)
                                bne        x1,loop        # Repeat if n != 0
                                vdisable

```

Disable vector registers